Genomic Mating R Pipeline

Expected-progeny-value optimization under an additive-dominance marker model

Updated 12 November 2026. Corresponds to **GM_Rcpp_single.R**, code release 2026-10-26.

What changed from the original R guide
The 19 existing arguments and their defaults are retained. The current single-file implementation adds `backend` and `search_tolerance`, compiles its embedded C++ code at loading, and uses exact Hungarian mating assignment by default. No separate .cpp file is needed.

Quick start

```
install.packages(c("Rcpp", "BGLR"))  # first installation only
source("GM_Rcpp_single.R")

dat <- read.csv("GM_ready_data.csv", check.names = FALSE)
res <- run_genomic_mating_pipeline(
  dat = dat,
  n_female_select = 60,
  n_male_select = 30,
  female_pool_size = 200,
  male_pool_size = 200,
  n_restart = 2,
  seed = 2026,
  search_mode = "auto",
  effect_estimator = "bglr",
  backend = "rcpp"
)
head(res$gm_results$pair_result_gm)
```

Install a working C++11 compiler before sourcing the file. Input must contain enough female and male candidates for the selected counts. The numbers above are an analysis template, not built-in sample data.

Pipeline at a glance

Input	Prediction	Optimization
ID, Sex, Phenotype; SNP dosage 0/1/2	Estimate additive and dominance effects; compute all female-by-male cross values	OSNB multistart parent exchange; exact assignment within each selected subset

Reading map: requirements and data (p. 2); arguments (p. 3); effect estimation and cross values (p. 4); optimization (p. 5); output object (p. 6); export and settings (p. 7); performance and troubleshooting (p. 8); reproducible script and methods wording (p. 9).

1. Software requirements and input data

Requirement	Purpose and availability
R and C++ toolchain	Use R 4.1 or later with C++11 compilation available. On Windows install the Rtools version matching your R installation; on Linux provide g++ and make; on macOS provide command-line developer tools.
Rcpp	Required when source("GM_Rcpp_single.R") is evaluated. The embedded C++ is compiled and loaded even if a later call selects the lpSolve backend.
BGLR	Required for effect_estimator="bglr", the default estimator. Called through BGLR::BGLR; library(BGLR) is not required.
MASS	Required for effect_estimator="em_reml" through MASS::ginv. MASS is normally supplied with a standard R installation.
lpSolve	Required only when the pipeline is called with backend="lpSolve". It is not required by the default Hungarian assignment path.

```
# Optional reference backend dependency:
install.packages("lpSolve")

# Direct compiler check:
Rcpp::cppFunction('int gm_check() { return 1; }')
stopifnot(gm_check() == 1)
```

Required data frame

There is one row per candidate parent. A plain `data.frame` is required; no pedigree, relationship matrix or map is passed to this end-to-end function.

Column	Required format
`ID`	Nonempty, nonmissing, unique identifier. Values are converted to character.
`Sex`	Exactly F or M; both sexes must be present. Values are converted to character.
`Phenotype`	Numeric and finite, with no missing values. One target trait per call.
`SNP...`	Column names start with SNP. Genotypes are numeric or coercible to numeric and must be 0, 1 or 2, with no missing values.

```
stopifnot(is.data.frame(dat))
stopifnot(all(c("ID", "Sex", "Phenotype") %in% names(dat)))
stopifnot(!anyDuplicated(dat$ID))
stopifnot(all(dat$Sex %in% c("F", "M")))
table(dat$Sex)
snp_cols <- grep("^SNP", names(dat))
stopifnot(length(snp_cols) > 0)
```

Extra columns are not automatically treated as covariates. Marker columns are identified only by their SNP prefix. Imputation or genotype recoding must be completed before calling the pipeline.

2. Function arguments and actual defaults

The function is still named `run_genomic_mating_pipeline()`. All 19 original arguments remain in the same order; the two Rcpp-specific arguments are appended.

Argument	Default	Meaning
`dat`	required	Input data.frame.
`n_female_select`	required	Number of dams selected; also the number of mating pairs.
`n_male_select`	required	Number of sires selected.
`female_pool_size`	200	Per-ranking female pool parameter, not a strict union-size cap.
`male_pool_size`	200	Per-ranking male pool parameter, not a strict union-size cap.
`n_restart`	2	Number of starts: first two are guided greedy starts.
`seed`	2026	Seed used for BGLR and the multistart search.
`force_use_all_males`	TRUE	Enforce balanced lower and upper male-use bounds when feasible.
`search_mode`	"best"	Alternatives: "first" or "auto". Set "auto" explicitly to use it.
`threshold`	1e-6	EM-REML convergence threshold, not the C++ search tolerance.
`maxit`	200	Maximum EM-REML iterations, not an OSNB iteration limit.
`verbose`	TRUE	Pipeline and search reporting.
`effect_estimator`	"bglr"	Alternative: "em_reml"; "em-reml" is accepted as an alias.
`bglr_model`	"BRR"	BGLR marker-effect model for additive and dominance components.
`bglr_nIter`	12000	BGLR total MCMC iterations.
`bglr_burnIn`	4000	BGLR burn-in iterations.
`bglr_thin`	5	BGLR thinning interval.
`bglr_verbose`	FALSE	BGLR console reporting.
`bglr_saveAt`	NULL	Temporary output prefix by default; supply a prefix to retain files.
`backend`	"rcpp"	Alternative: "lpSolve" reference implementation.
`search_tolerance`	1e-12	Nonnegative C++ improvement/tie tolerance; reference path uses zero.

Selected counts, pool parameters, n_restart and maxit must be positive integers. Each pool parameter must be at least its selected-parent count; the input must contain enough parents of each sex. Defaults are those selected by match.arg when vector-valued defaults are omitted.

3. Effect estimation and expected cross values

Marker coding

Genotype	Additive dosage Z	Dominance code W = I(Z = 1)
aa	0	0
Aa	1	1
AA	2	0

The pipeline fits additive and dominance marker coefficients, denoted gamma_hat and delta_hat. Each candidate parent is scored as the sum of Z * gamma_hat + W * delta_hat over the SNPs. This is total marker-predicted genetic value, not an additive-only estimated breeding value.

Default: BGLR

```
res_bglr <- run_genomic_mating_pipeline(
  dat, n_female_select = 60, n_male_select = 30,
  effect_estimator = "bglr", bglr_model = "BRR",
  bglr_nIter = 12000, bglr_burnIn = 4000, bglr_thin = 5,
  search_mode = "auto", backend = "rcpp"
)
```

The additive dosage matrix is supplied to one BGLR component. A second heterozygosity component is included when its design has variation; otherwise dominance coefficients are set to zero. When bglr_saveAt is NULL, a temporary prefix is used and its files are removed on exit. A user-supplied prefix retains BGLR output files.

Alternative: internal EM-REML

```
res_em <- run_genomic_mating_pipeline(
  dat, n_female_select = 60, n_male_select = 30,
  effect_estimator = "em_reml", maxit = 200,
  threshold = 1e-6, search_mode = "auto", backend = "rcpp"
)
```

The EM-REML path estimates variance components and recovers marker effects using generalized inverses from MASS. Its iteration history is returned in variance_components$results. Changing the search backend does not change which effect estimator is requested.

Mendelian expected progeny value

For dam i and sire j at SNP k, let u = $Z_{ik}/2$ and v = $Z_{jk}/2$. Then $E(Z_{ij,k}) = u + v$ and $E(W_{ij,k}) = u + v - 2uv$.

```
xi_ij = sum_k [ E(Z_ij,k) * gamma_hat_k
              + E(W_ij,k) * delta_hat_k ]
```

Interpret the scale correctly
The cross matrix and optimized objective contain the marker genetic score, without an added phenotype intercept. They should not be presented as absolute predicted phenotypic means. The objective is the SUM over selected crosses; divide by the number of pairs to obtain their mean.

4. Two-level OSNB optimization

The upper level uses multistart single-parent exchange to search parental subsets. For each subset, the lower level finds the exact optimal mating assignment under the prescribed male-use bounds. Exact conditional assignment does not certify a globally optimal parental subset.

Stage	Current implementation
Candidate ranking	One space ranks females by row means and males by column means of the full cross-value matrix. Another ranks parents by their own predicted total genetic values.
Search pool	The upper search uses the union of the two ranking-based spaces for each sex. A pool parameter of 200 applies to each ranking, so the union may contain more than 200 candidates per sex; it is capped by the available input population.
First two starts	Start 1 is value-greedy; start 2 is parent-greedy. They need not be distinct and are not two independent random starts. Additional starts alternate between randomized value-space and parent-space starts.
Parent exchange	Replace one selected female or male with an unselected candidate from the union. Reoptimize allocation for the proposed subset and accept an improving move under the selected search mode.
Default lower solver	C++ exact Hungarian assignment with sire slots; the C++ local-search kernel is also used. The nested function name solve_mating_ilp_general is retained for compatibility.
Reference backend	backend="lpSolve" uses the R search path and lpSolve binary ILP. It represents the same balanced assignment objective/constraints; numerical ties can yield different equivalent pairings.

Search mode

Mode	Rule
`best`	Evaluate all current 1-swap neighbors and accept the best improvement. This is the actual default.
`first`	Accept the first improving neighbor in the implementation order.
`auto`	Choose best or first using the original neighborhood-count and assignment-size workload heuristic. The heuristic retains legacy ILP-size labels even on the Rcpp backend.

Mating constraints

Every selected dam is assigned once. With force_use_all_males=TRUE and at least as many dams as sires, each selected sire has a load between floor(n_female_select/n_male_select) and ceiling(n_female_select/n_male_select). For 60 dams and 30 sires, each sire is used exactly twice. If there are fewer dams than sires, not all selected sires can receive a mating. With FALSE, the lower load is zero but the capacity upper bound remains.

The Rcpp search_tolerance defaults to 1e-12 and is separate from the EM-REML threshold. Retain the default for routine runs and record any change.

5. Output object and result interpretation

Output component	Contents
backend	Actual backend selected: rcpp or lpSolve.
implementation	Code version, lower solver and upper solver descriptions.
run_settings	Seed, restart count, requested search mode, selected counts, pool parameters and male-use setting.
search_tolerance	Effective C++ tolerance; zero for the lpSolve reference path.
timings	Elapsed seconds for effect_estimation, value_matrix, optimization and total.
effect_estimator	The estimator used: bglr or em_reml.
bglr_settings	BGLR settings when used; otherwise NULL.
variance_components	Estimator output/history and sigmaE, sigmaA, sigmaD, kA and kD.
marker_effects	SNP, gamma_hat and delta_hat.
parent_genetic_values	ID, Sex, Phenotype and total marker-predicted GeneticValue.
expected_progeny_value_matrix	Full female-by-male expected genetic score matrix.
gm_results$fit_gm	Best objective, selected sets, assignment, sire load and multistart history.
gm_results$pair_result_gm	Final Dam/Sire pairs with ExpectedProgenyValue, DamGeneticValue and SireGeneticValue.

Inspect the optimized design

```
pairs <- res$gm_results$pair_result_gm
head(pairs)
res$gm_results$fit_gm$best_final_objective
res$gm_results$fit_gm$sire_load
res$gm_results$fit_gm$history
res$gm_results$fit_gm$global_search_mode
res$implementation
res$timings
```

Timing boundaries
The total pipeline timer starts when run_genomic_mating_pipeline() is called. Compilation occurs during source("GM_Rcpp_single.R") and is outside these returned timings. For end-to-end wall time, time both sourcing and the function call. Timings are in seconds.

BGLR sigmaA/sigmaD summaries in this implementation are variances of the fitted additive and dominance score vectors, with a positive floor; they are not interchangeable with all other variance estimators or with predicted offspring segregation variance.

6. Exporting results and selecting settings

```
write.csv(res$gm_results$pair_result_gm,
          "GM_pair_result.csv", row.names = FALSE)
write.csv(res$marker_effects,
          "GM_marker_effects.csv", row.names = FALSE)
write.csv(res$parent_genetic_values,
          "GM_parent_genetic_values.csv", row.names = FALSE)
write.csv(res$expected_progeny_value_matrix,
          "GM_expected_progeny_value_matrix.csv")
write.csv(res$gm_results$fit_gm$history,
          "GM_multistart_history.csv", row.names = FALSE)
saveRDS(res, "GM_full_result.rds")
writeLines(capture.output(sessionInfo()), "GM_sessionInfo.txt")
```

Basic consistency checks

```
pairs <- res$gm_results$pair_result_gm
stopifnot(nrow(pairs) == res$run_settings$n_female_select)
stopifnot(!anyDuplicated(pairs$Dam))
stopifnot(abs(sum(pairs$ExpectedProgenyValue) -
  res$gm_results$fit_gm$best_final_objective) < 1e-7)
mean(pairs$ExpectedProgenyValue)
table(pairs$Sire)
```

Settings by use case

Use case	Suggested approach
Initial code check	Use fewer selected parents and smaller input data. A short BGLR chain can check execution, but does not demonstrate convergence or establish final effect estimates.
Routine empirical analysis	Use backend="rcpp", n_restart=2, pool parameters near 200 when compatible with selected counts, and explicitly choose search_mode="auto". Check effect-estimation diagnostics.
Quality-focused search	Use search_mode="best" and investigate additional restarts. Upper-level convergence is local, so extra computation does not prove global optimality.
Larger candidate groups	Use auto or first as appropriate. Candidate pool parameters act after the full cross-value matrix is built. Any input prescreening changes the candidate population and must be reported.
Backend comparison	Use the same data, estimator, seed, parent counts and mode. Compare objective values with numerical tolerance; equivalent tied assignments need not have identical pair IDs.

Do not treat n_restart=2 as two independent random starts. Do not describe pool_size=200 as a strict total of 200 parents, or maxit as an upper-level search budget.

7. Performance, caching and troubleshooting

The full female-by-male matrix is computed before parental screening. With nF females, nM males and m SNPs, direct cross-value calculation scales with nF * nM * m; storing a double-precision nF-by-nM matrix alone requires about 8 * nF * nM bytes, excluding other objects. Rcpp accelerates cross scoring, local exchange and assignment, but does not remove effect-estimation costs.

Compilation cache

The source uses Rcpp::sourceCpp with rebuild=FALSE. The default cache is inside the current R session temporary directory. To reuse compiled artifacts across sessions, set a writable cache directory before sourcing. Keep separate caches for incompatible R/compiler installations.

```
Sys.setenv(GM_RCPP_CACHE = file.path(path.expand("~"), "gm_rcpp_cache"))
source("GM_Rcpp_single.R")
```

Common problems

Problem	What to check
Rcpp package missing	Install Rcpp before source(). Loading the file, including for a later lpSolve call, requires Rcpp.
Compilation fails	Confirm an installed C++11 toolchain. On Windows use matching Rtools, restart R after installation and run the compiler check on page 2. Check cache write permissions.
No SNP columns found	Rename marker columns so every genotype column starts with SNP.
Invalid input values	ID must be unique, nonempty and nonmissing; Sex must be F/M; Phenotype must be finite numeric; SNP dosage must be 0/1/2 without missing values.
Too few candidates	Selected counts must not exceed available parents by sex. Each pool parameter must be at least the corresponding selected count.
BGLR files not retained	Supply a writable bglr_saveAt prefix. With NULL, temporary BGLR files are automatically removed.
MASS or lpSolve error	MASS is needed for EM-REML; lpSolve is needed for the reference backend. Install a version compatible with the installed R.
Slow despite Rcpp	Separate effect-estimation, full-matrix and optimization times. Check actual union sizes and chosen mode; best can evaluate many assignments.
Different equivalent pairs	Ties, near-ties, solver ordering and numerical tolerance can change pair identities. Compare valid loads and objective values before concluding that the models differ.

Rcpp is a compiled implementation, not another name for lpSolve. Hungarian and binary ILP are different algorithms that solve the same specified balanced assignment subproblem.

8. Reproducible script and reporting

```
# Requires Rcpp, BGLR and a working C++11 compiler.
source("GM_Rcpp_single.R")
dat <- read.csv("GM_ready_data.csv", check.names = FALSE)

res <- run_genomic_mating_pipeline(
  dat = dat,
  n_female_select = 60, n_male_select = 30,
  female_pool_size = 200, male_pool_size = 200,
  n_restart = 2, seed = 2026,
  force_use_all_males = TRUE, search_mode = "auto",
  effect_estimator = "bglr", bglr_model = "BRR",
  bglr_nIter = 12000, bglr_burnIn = 4000, bglr_thin = 5,
  bglr_verbose = FALSE,
  backend = "rcpp", search_tolerance = 1e-12
)
write.csv(res$gm_results$pair_result_gm,
          "GM_pair_result.csv", row.names = FALSE)
saveRDS(res, "GM_full_result.rds")
writeLines(capture.output(sessionInfo()), "GM_sessionInfo.txt")
```

Suggested Methods wording

Genomic mating was implemented under an additive-dominance marker model using the Rcpp pipeline. SNP dosage was coded as 0, 1 and 2, and dominance as a heterozygote indicator. Marker effects were estimated using [specify BGLR and its settings, or internal EM-REML]. Expected progeny genetic values were calculated from Mendelian expectations of additive and dominance codes. The OSNB upper level searched parental subsets using multistart single-parent exchange, while an exact Hungarian slot-assignment algorithm optimized mating allocation conditional on each subset. The objective was the sum of expected progeny genetic values across the selected mating pairs.

Report the input size by sex, SNP number, selected-parent counts, pool parameters and actual union sizes where relevant, restart count, requested and selected search modes, estimator settings, seed, backend, search tolerance and software/compiler versions. Distinguish conditional assignment optimality from heuristic parental selection and identify timing boundaries.

Version and compatibility

This guide matches the single-file Rcpp implementation released 2026-10-26. It preserves the legacy public arguments and output fields and adds backend metadata and timing fields. A small-data compatibility check covered BGLR and the internal EM-REML path; this does not guarantee identical stochastic estimates or tied pairings on every platform or dataset.

Software resources

Repository: GM_Rcpp_single.R
CRAN: Rcpp | CRAN: BGLR | Rtools for Windows